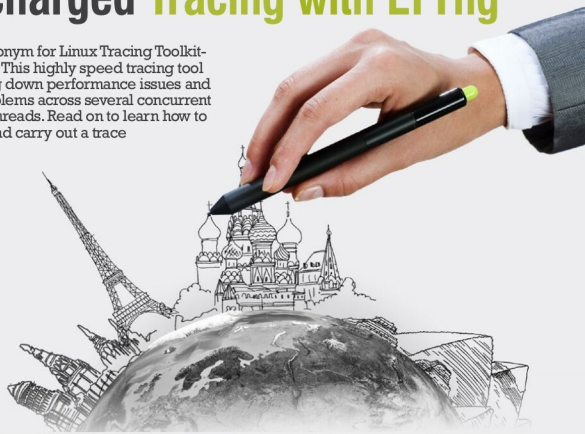
Turbocharged Tracing with LTTng

LiTTng is the acronym for Linux Tracing Toolkit-next generation. This highly speed tracing tool helps in tracking down performance issues and debugging problems across several concurrent processes and threads. Read on to learn how to install the tool and carry out a trace



Tracing in the Linux world is part of the performance analysis activities such as debugging, profiling or even logging. Tracers are generally quicker and more accurate than other tools used for performance analysis. But why do we need them?

Tracing 101

Consider a soft real-time system in which the correctness of the output is highly dependent on not just the accuracy, but also on how long a program takes to execute in it. In such a system, it's not really feasible to use the traditional debug approach of pausing the program and analysing it. Even a small *ptrace()* call can add an unwanted delay to the whole execution. Simultaneously, there could be a need to gather huge amounts of data from the kernel as well as your user space application at the same time. Is there a way to gather all that without disrupting the program's execution?

Indeed, there is. The answer to all these questions is a technique called tracing, which is more like system-wide logging, but at a very low and fine grained level. Getting such details is particularly helpful where intricate, time accurate and well represented details of the system's functioning cannot be achieved by traditional debuggers like GDB or KGDB. Neither can sampling-based profiling tools such as Perf prove to be completely useful.

Tracing can be divided according to the functional aspect (static or dynamic) or by its intended use (kernel or userspace tracing–also known as *tracing domains*, in the case of LTTng). Static tracing requires source code modification and recompilation of the target binary/kernel, whereas in dynamic tracing, you can insert a tracepoint directly into a running process/kernel or in a binary residing on the disk.

Before we move further, let's understand some jargon and see what most of the tracing tools do. Tracing usually involves adding a special tracepoint in the code. This tracepoint can look like a simple function call, which can be inserted anywhere in the code (in case of userspace applications) or be provided as part of standard kernel tracing infrastructure (tracepoint 'hooks' in the Linux kernel). Each tracepoint hit is usually associated with an event. The events are very low level and occur frequently. Some examples are syscall entry/exit, scheduling calls, etc. For userspace applications, these can be your own function calls. In general, tracing involves storing associated data in a special buffer whenever an event occurs. This data is obviously huge and contains precise timestamps of the tracepoint hit, along with any optional event-specific information (value of variables, registers, etc). All this information can be stored in a specific format for later retrieval and analysis.

Tracing tools

An important point to note is that not all available tracing tools